EU-Flex Model User Manual

Version 1.0

Table of Contents

EU-Flex Model User Manual 4

- 1 Installation Guide 5
  - 1.1 Install Prerequisites 5
  - 1.2 Get the Model 7
  - 1.3 Set up the Environment 8
- 2 Running the Model 9
  - 2.1 Quick Start 9
  - 2.2 What the Workflow Does 10
  - 2.3 Wildcards 11
  - 2.4 Choosing What to Run 11
  - 2.5 Useful Snakemake Options 12
  - 2.6 Working Directories 13
  - 2.7 Reproducing the ACER Paper 14
  - 2.8 Exploring the Results in a Notebook 16
- 3 Configuration 18
  - 3.1 How Configuration Works 18
  - 3.2 Run and Scenario 19
  - 3.3 Network Construction 20
  - 3.4 Carriers and Unit Commitment 21
  - 3.5 Market Interconnection 22
  - 3.6 Reserves 23
  - 3.7 Demand-Side Response 24
  - 3.8 Storage Planning 24
  - 3.9 Dummy Storage Optimisation 25
  - 3.10 Forced Outages 26
  - 3.11 Maintenance Planning 27
  - 3.12 Price Caps 28
  - 3.13 Negative Pricing 28
  - 3.14 The EVA Loop 29
  - 3.15 Costs and Net Present Value 30
  - 3.16 Flexibility Indicators 31
  - 3.17 Market Clearing and Solver 32
  - 3.18 Temporal Clustering 33
  - 3.19 Perfect Foresight and Infeasibility Handling 34

- 3.20 Outputs and Logging 34
- 3.21 Environment Variables 35
- 4 Intermediate Files 36
- 5 Model Outputs 37
  - 5.1 File Structure 37
  - 5.2 Variable Groups 39
  - 5.3 Accessing the Data 42
- 6 Output Directory Structure 43
- 7 Worked Examples 44
  - 7.1 Summary Plots 44
  - 7.2 With the Dummy Storage Optimisation 45
  - 7.3 Reserves Off, To Isolate Their Effect 45
  - 7.4 Sensitivity Mode 45
- 8 Troubleshooting 46
  - 8.1 Missing Input Files 46
  - 8.2 Configuration Rejected at Startup 47
  - 8.3 Infeasible Market Windows 48
  - 8.4 The Run Is Too Slow 48
  - 8.5 Restarting an Interrupted Run 48
  - 8.6 Running Offline 49
  - 8.7 Reproducibility 49
  - 8.8 Inspecting What Will Run 50
- 9 Feedback 50
  - How to provide feedback 50
  - What feedback is collected here 51
- 10 Licence and Contributions 51

EU-Flex: Flexibility Assessment Model of the Future European Energy System

The EU-Flex Model is a high-resolution market simulation of the European Energy system, which is built on top of the open energy system modelling framework PyPSA for which documentation is available as well.

The EU-Flex Model has been developed by ACER with the support of Open Energy Transition (OET) for the flexibility needs assessment in 2027.

Note that the documentation below contains the User Manual for the public EU-Flex Model.

EU-Flex Model User Manual

The EU-Flex model is an open-source implementation of ACER's EU-wide flexibility assessment methodology, built on top of PyPSA-Eur. It evaluates power system flexibility needs across the European Union, as mandated by Regulation (EU) 2024/1747.

The model is a Snakemake workflow. You run it with the `snakemake` command and control it with a YAML configuration file. It builds a network for each target year and climate year, simulates day-ahead and intraday markets over an iterative Economic Viability Assessment (EVA), and writes a set of flexibility indicators.

What is included in this release. The release contains the model code, its configuration options, and the test model that accompanies the ACER paper "Preparing for the European flexibility needs assessment" (ACER, 2026), called *the ACER Paper* in this manual. The ACER Paper model is in `data/teaser/`, with its four configurations in `config/teaser.yaml`, `config/teaser-eva.yaml`, `config/teaser-ds-cross-border.yaml` and `config/teaser-ds-no-cross-border.yaml` (see 2.1).

The README in the root of the repository has the short version: step-by-step installation and the commands to run the ACER Paper model. This manual covers the same ground in more detail, and documents every configuration option the model accepts. The mathematical formulation of the model is documented separately in the EU-Flex Modelling Specification.

---

1 Installation Guide

Three steps are needed before you can run anything:

1. Install prerequisites: the required software.

2. Get the model: download the source.

3. Set up the environment: create and activate the Python environment.

1.1 Install Prerequisites

The model runs on these platforms:

Platform	Lock file	Tested
Windows 10 and 11, x86-64 (Intel or AMD)	`envs/win-64.lock.yaml`	yes
Linux, x86-64	`envs/linux-64.lock.yaml`	yes
macOS, Apple silicon (M1 and later)	`envs/osx-arm64.lock.yaml`	yes
macOS, Intel	`envs/osx-64.lock.yaml`	no

Tested means that a short synthetic test case is installed and run on that platform before a release, triggered manually through the CI system. Windows on ARM and Linux on ARM are not supported.

Which terminal. Every command in this manual is the same on every platform, unless it is shown separately for Windows. On Windows, run the commands in the *Miniforge Prompt* (or, with Miniconda or Anaconda, the *Anaconda Prompt*), which the conda installer adds to the Start menu. It has conda ready to use, and the commands work there unchanged. PowerShell works too, but first needs conda set up for it: run `conda init powershell` once in the Miniforge Prompt and open a new PowerShell window. If PowerShell then reports that running scripts is disabled, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` in PowerShell and open a new window. On Linux and macOS, use any terminal.

1.1.1 Git

Git is a version control system. You need it to obtain the source code. On Windows, install Git for Windows with its default options. On macOS, run `git --version` in a terminal: if Git is not installed, macOS offers to install it. On Linux, install it with your package manager, for example `sudo apt install git`. More detail is on the official Git website.

1.1.2 Conda

The model depends on a specific set of Python and solver packages, managed with conda. We recommend Miniforge, which ships conda with the `conda-forge` channel preconfigured.

- **Windows:** download `Miniforge3-Windows-x86_64.exe` from the Miniforge download page and run it, accepting the defaults.
- **Linux and macOS:** run these two commands in a terminal and answer `yes` to the questions, then close the terminal and open a new one:

```
curl -L -O "https://github.com/conda-forge/miniforge/releases/
latest/download/Miniforge3-$(uname)-$(uname -m).sh"
bash Miniforge3-$(uname)-$(uname -m).sh
```

If you already have Miniconda or Anaconda, you can use it instead and skip this step. The environment is installed from lock files that name the exact package builds, so it comes out the same whichever conda you use. Micromamba, a small standalone alternative, also works; use `micromamba` in place of `conda` in the commands below.

To check that conda works, open a new Miniforge Prompt (Windows) or terminal (Linux, macOS) and run `conda --version`. It prints a version number.

1.1.3 A Solver

The model ships configured for HiGHS, which is open source and installed with the environment, so you do not need to do anything extra to get started. Gurobi is also supported and is noticeably faster on the larger cases, but it is commercial and needs to be installed separately (with its own licence). Other solvers (CBC, which is open source, and the commercial CPLEX, Xpress and Knitro) can be selected, but they come without bundled option sets and are not routinely tested. See 3.17 Market Clearing and Solver.

1.2 Get the Model

On Windows, clone the model to a short path such as `C:\eu-flex-model`, since Windows limits paths to 260 characters by default:

```
cd C:\
git clone https://github.com/EU-ACER/eu-flex-model.git eu-flex-model
cd eu-flex-model
```

On Linux and macOS, clone it wherever you like, for example in your home folder:

```
git clone https://github.com/EU-ACER/eu-flex-model.git eu-flex-model
cd eu-flex-model
```

This creates a folder `eu-flex-model` holding the model code and its input data, including the ACER Paper model. This manual calls it the *checkout*. Run all commands from its root unless a section says otherwise.

1.3 Set up the Environment

Install from the lock file for your platform (see the table in 1.1). These pin every package to an exact version and build, so everyone gets the same environment and a new upstream release cannot change your results. Creating the environment downloads the packages the model needs and can take several minutes.

```
:: Windows
conda env create -f envs/win-64.lock.yaml
```

```
# Linux
conda env create -f envs/linux-64.lock.yaml
```

```
# macOS, Apple silicon
conda env create -f envs/osx-arm64.lock.yaml
```

```
# macOS, Intel
conda env create -f envs/osx-64.lock.yaml
```

The lock files are generated with conda-lock from `envs/environment.yaml`, which lists the direct dependencies and their version ranges.

Then, on every platform, activate the environment and install the model itself into it:

```
conda activate eu-flex
pip install -e .
```

`conda activate eu-flex` lasts for the current terminal only. Run it again in every new terminal before you use the model.

Verify the installation by asking Snakemake to plan a run of the ACER Paper model without executing it:

```
snakemake -n -c1 --configfile config/teaser.yaml --directory runs/
teaser
```

This lists the jobs it would run, followed by a table of job counts per rule and a few notes from Snakemake (for example that the workflow contains checkpoints, which add jobs while it runs, so the plan is not final).

If it gets that far without an error, you are ready. An error instead, such as a `MissingInputException`, usually means a command was not run from the root of the checkout; see 8.

2 Running the Model

2.1 Quick Start

The ACER Paper model is the test model behind the results of the ACER Paper. It is a stylised three-node system (buses `N1`, `N2` and `N3`) for target year 2030 and climate year 2013, covering a full year of hourly snapshots. It is stored in `data/teaser/ty_2030_cy_2013/` as a folder of PyPSA component CSVs (`buses.csv`, `generators.csv`, `storage_units.csv`, the timeseries, …), which is the format sensitivity mode reads, together with the two day-ahead forecast files `generators-p_max_pu-forecast.csv` and `loads-p_set-forecast.csv`. The capacities are the state before the EVA: they have not been iterated to convergence.

The ACER Paper uses three ways of running the model, each with its own configuration:

1. `config/teaser.yaml` computes the flexibility indicators on the network as supplied, with a single EVA iteration and without the dummy storage optimisation.

2. `config/teaser-eva.yaml` first iterates the capacities through the EVA, for up to 30 iterations with a coarse first step that decays linearly, and then computes the indicators on the result.

3. `config/teaser-ds-cross-border.yaml` and `config/teaser-ds-no-cross-border.yaml` run a single iteration without maintenance planning, followed by the dummy storage optimisation: for each of 7 storage durations and 9 curtailment targets, it sizes the storage needed to meet the target. In the first, cross-border link flows are re-optimised together with the dummy storage. In the second, they are fixed to the market result.

Run them from the root of the checkout, with the `eu-flex` environment active:

```
snakemake -c all --configfile config/teaser.yaml --directory runs/
teaser
snakemake -c all --configfile config/teaser-eva.yaml --directory
runs/teaser-eva
snakemake -c all --configfile config/teaser-ds-cross-border.yaml --
directory runs/teaser-ds-cross-border
snakemake -c all --configfile config/teaser-ds-no-cross-border.yaml
--directory runs/teaser-ds-no-cross-border
```

The commands are the same on Windows, Linux and macOS. `-c all` uses every available core. `--directory` gives each run a folder of its own under `runs/` in the checkout, its *working directory*,

so that the runs do not overwrite each other's output. Each writes its results to `results/` there, intermediate files to `resources/`, and logs to `logs/` (see 6); `runs/` is gitignored, so a run never shows up as a change to commit. The runs are independent: you can start one in another terminal while another is running, as long as the machine has the cores and memory for both.

How long a run takes depends on the machine, so take these as orders of magnitude. The first run takes in the order of ten minutes. In the second, each iteration does about as much work as the whole first run, so with up to `eva.max_it: 30` iterations it can take several hours; it ends sooner if the capacities converge earlier (see 3). Each dummy storage run solves 63 optimisations (7 storage durations × 9 curtailment targets) of a few minutes each, about three and a half hours of computing in total, so with a few running in parallel it takes in the order of an hour. Each optimisation needs up to 2 GB of memory, so on a machine with little memory use a lower `-c`, such as `-c 4`, to run fewer at once. Because the configurations set `eva.dummy_storage_optimization.export_models: true`, a dummy storage run also writes about 9 GB of `.lp` model files.

While a run is going, Snakemake prints each job as it starts and finishes, with a running count such as `12 of 20 steps (60%) done`. In the economic viability assessment Snakemake only adds the jobs of an iteration once the previous one has finished, so the total grows with every iteration and the percentage drops back each time. A run is finished when the command returns; a run that completes ends with a count at 100%. Its main result is `results/assessment/model_outputs.nc` in its working directory, such as `runs/teaser/results/assessment/model_outputs.nc`, which holds every indicator; see 5 for what is in it and how to open it.

To change a setting, edit the configuration file, or copy it under a new name and edit the copy (see 3). Give a changed run its own `--directory`, or it overwrites the results already there.

> The model fetches no data at run time. Everything it needs is in `data/`, so it runs offline out of the box. See 8.6.

2.2 What the Workflow Does

For every combination of target year and climate year, the model:

1. **Builds a base network** from the input network, applying your configuration.
2. **Plans maintenance** outages across the year for thermal units.
3. **Samples forced outages** stochastically.
4. **Plans storage** at a coarse time resolution to set reservoir trajectories.
5. **Clears the day-ahead market** on forecast timeseries, over a rolling horizon.
6. **Clears the intraday market** on actual timeseries, anchored to the day-ahead result.

7. **Computes the net present value** of every asset from its market revenues.

8. **Updates capacities**: assets that earn enough are expanded, assets that do not are decommissioned or mothballed.

Steps 1 to 8 repeat until capacities stop changing or `eva.max_it` is reached. This loop is the EVA, and the `eva` section of the configuration is named after it. Forced outages are sampled once; maintenance and storage plans are re-computed only every N iterations (see `eva.maintenance_plan_frequency` and `eva.storage_planning.storage_plan_frequency` in 3.14 and 3.8). The loop has converged once no asset changes capacity. The final iteration then feeds the flexibility indicator calculations and, optionally, the dummy storage optimisation.

2.3 Wildcards

Snakemake identifies each job by the wildcards in its filenames. You will see these throughout the logs and in `resources/`:

- `ty` is the target year, the future year being modelled. Which years are available depends on the input networks; the ACER Paper model has 2030 only.
- `cy` is the climate year, the historical weather year driving renewable generation and demand. Separate networks are built for every `ty` x `cy` combination. The ACER Paper model has 2013 only.
- `ft` is the timeseries type, either `forecast` (day-ahead) or `actual` (intraday).
- `fo` is the forced outage sample, for example `fo1`. There is one per sample configured.
- `it` is the EVA iteration number, counting from 0.
- `mh` is the energy-to-power ratio in hours of a dummy storage unit, used only in the dummy storage optimisation.
- `ct` is the curtailment target scenario, also dummy storage only.

2.4 Choosing What to Run

The workflow is made up of *rules*, each a step that turns input files into output files. You tell Snakemake what you want, a *target*: either the name of a rule or a file the workflow produces. Snakemake then works out which rules need to run to produce it, skipping any whose outputs already exist and are up to date. It also tracks changes to input files and code, and reruns the steps downstream of a change that could affect the results.

Without a target, Snakemake builds everything, including the flexibility indicators. These examples run the ACER Paper model, as in 2.1:

```
# Everything (the default)
snakemake -c all --configfile config/teaser.yaml --directory runs/
teaser

# Only the market simulations, skipping the assessment
snakemake -c all --configfile config/teaser.yaml --directory runs/
teaser -- solve_eva_networks

# A single intermediate file, relative to the working directory
snakemake -c all --configfile config/teaser.yaml --directory runs/
teaser -- resources/networks/base_network_2030_2013_actual_0.nc
```

The useful rule targets are:

Target	What it produces
`all`	Everything: the default when no target is given.
`solve_eva_networks`	The market simulations of every EVA iteration, up to the final networks in `results/networks/`.
`run_flexibility_assessment`	The flexibility indicators and other outputs, in `results/assessment/model_outputs.nc` (see 5), without the summary plots.

When you pass `--configfile`, separate the targets from the options with `--`. `--configfile` accepts several files, so without the `--` a target after it is read as another configuration file:

```
snakemake -c all --configfile config/teaser.yaml --
solve_eva_networks   # works
snakemake -c all --configfile config/teaser.yaml
solve_eva_networks      # fails
```

2.5 Useful Snakemake Options

Option	What it does
`-n` / `--dry-run`	Lists the jobs that would run without running them. Always worth doing first.
`-c N` / `--cores N`	Runs up to N jobs in parallel. `-c all` (or just `-c`) uses every core.
`--directory DIR`	Runs in DIR instead of the current directory. Outputs go there.

Option	What it does
`--configfile FILE`	Loads an extra configuration file on top of the defaults.
`--config key=value`	Overrides a single value from the command line.
`--rerun-incomplete`	Redoes jobs left half-finished by an interrupted run.
`--forcerun RULE`	Forces a rule to re-run even if its outputs look current.
`--keep-going`	Carries on with independent jobs after one fails.
`--dag | dot -Tpng -o dag.png`	Draws the job dependency graph. Needs Graphviz; see 8.8.

2.6 Working Directories

Everything a run writes goes to its working directory: the folder named by `--directory`, or the folder you run `snakemake` from if you leave it out. A second run in the same working directory overwrites the first, so give each run a working directory of its own.

The simplest place for working directories is `runs/` in the checkout, as in 2.1. From there the model finds the checkout, and with it the input data in `data/`, by searching upwards, so nothing else needs setting up. Relative paths in a configuration file are looked up differently depending on the option: `eva.build_network.sensitivity_path`, forecast CSV paths (`eva.build_network.forecast_p_max_pu_path` and `eva.build_network.forecast_p_set_path`), the maintenance plan path (`solving.maintenance_planning.csv_path`) and FBMC data paths are looked up first in the working directory and then in the checkout; others, such as `eva.build_network.workflow_prebuilt.path`, are looked up in the checkout only (or under `EUFLEX_INPUT_DIR`, see below). The path given to `--configfile` itself is different: Snakemake looks it up from the folder you run `snakemake` in, which is why the commands in this manual are run from the root of the checkout.

A working directory can also be outside the checkout. The model then cannot find the checkout by searching upwards, and needs to be told where it is with the `EUFLEX_INPUT_DIR` environment variable, set to the root of the checkout. From the root of the checkout:

```
# Linux and macOS
export EUFLEX_INPUT_DIR=$PWD
snakemake -c all --configfile config/teaser.yaml --directory /path/
to/my-run
```

```
:: Windows, Miniforge Prompt
set EUFLEX_INPUT_DIR=%CD%
```

```
snakemake -c all --configfile config/teaser.yaml --directory C:
\path\to\my-run
```

In PowerShell, set the variable with `$env:EUFLEX_INPUT_DIR = $PWD`. The variable only lasts for the current terminal, so set it again in a new one. Write the path to `--directory` out in full: in PowerShell, `~` is not expanded in arguments to programs such as `snakemake` (see 8.1).

If a working directory contains a file `config/config.yaml`, the model reads it on every run there, so you can leave out `--configfile` (see 3.1).

2.7 Reproducing the ACER Paper

The four configurations of 2.1 are the settings of the runs behind the ACER Paper. The flexibility indicators of the paper come from `runs/teaser/results/assessment/model_outputs.nc`, the EVA results from `runs/teaser-eva/results/assessment/model_outputs.nc`, and the dummy storage results from the `/dummy_storage_optimization` group of `model_outputs.nc` in `runs/teaser-ds-cross-border/` and `runs/teaser-ds-no-cross-border/` (see 5). To reproduce the figures of the paper on the results of the first run, use the notebook of 2.8.

Each configuration is complete on its own: it sets everything that differs from the bundled defaults. The settings that matter for the ACER Paper are:

Option	`teaser.yaml`	`teaser-eva.yaml`	Why
`eva.sensitivity`	`true`	`true`	Read the network from the CSVs in `data/teaser/` rather than from a pre-built NetCDF file.
`eva.build_network.sensitivity_path`	`data/teaser/ty_{ty}_cy_{cy}`	same	Where the CSVs are (see 3.3).
`eva.build_network.forecast_timeseries_source`	`csv`	`csv`	Use the day-ahead forecasts supplied with the model rather than synthetic noise.
`eva.max_it`	`1`	`30`	One iteration computes the indicators on the network as supplied; thirty let the EVA iterate the capacities.

Option	teaser.yaml	teaser-eva.yaml	Why
eva.capacity_update.*	defaults	threshold: 20000, increment: 3000, linear_decay_iterations: 20, linear_decay_min_step: 100	The step sizes of the EVA (see 3.14).
eva.short_term_requirements.method	simulation	simulation	Short-term needs from the difference between the intraday and day-ahead markets (see 3.16).
eva.dummy_storage_optimization.enable	false	false	The dummy storage optimisation has runs of its own (below).
eva.storage_planning.po_resolution	4h	4h	Time resolution of the storage pre-optimisation.
eva.ramping_shortfall.enable	false	false	The ACER Paper does not compute the ramping shortfall indicator, which re-runs the final intraday market without ramping limits.
solving.solver_options.highs-default.ipm_optimality_tolerance	1e-6	1e-6	A tighter optimality tolerance for HiGHS than the default 1e-4.

The two dummy storage configurations share the settings of teaser.yaml, including eva.max_it: 1, and add:

Option	teaser-ds-cross-border.yaml	teaser-ds-no-cross-border.yaml	Why
solving.maintenance_planning.enable	false	false	No planned maintenance outages, so maintenance does not drive the storage needs. Forced outages still apply.
eva.dummy_storage_optimization.enable	true	true	Run the dummy storage optimisation after the iteration.
eva.dummy_storage_optimization.energy_power_ratios	[2, 4, 8, 20, 50, 100, 200]	same	The storage durations, in hours.
eva.dummy_storage_optimization.curtailment_targets	[0.09, 0.08, ..., 0.01]	same	The curtailment targets, from 9% down to 1%.

Option	`teaser-ds-cross-border.yaml`	`teaser-ds-no-cross-border.yaml`	Why
`eva.dummy_storage_optimization.shared_across_countries`	`true`	`false`	Cross-border link flows are re-optimised with dummy storage, or fixed to the market result.
`eva.dummy_storage_optimization.minimize_system_costs`	`false`	`false`	Minimise the dummy storage capacity rather than the total system cost.
`eva.dummy_storage_optimization.network_needs`	`false`	`false`	Leave network needs out.
`eva.dummy_storage_optimization.enable_load_spillage`	`true`	`true`	Keep the optimisation feasible at the tightest targets.
`eva.dummy_storage_optimization.conventional_floor_carriers`	`[Nuclear]`	same	Of the technical minimums of conventional generators, only that of nuclear limits how far conventional output can drop; must-run requirements always apply.
`eva.dummy_storage_optimization.export_models`	`true`	`true`	Write the `.lp` file of each optimisation, next to its network in `results/assessment/dummy_storage_optimization/`.

Everything else is read from the network CSVs. The run covers the full year in the files. The ACER Paper configurations also set `run.keep_files: true`, so no intermediate files are discarded; `eva.ramping_shortfall.enable: false`, so the `intraday_network_without_ramping` file is not computed; and `eva.build_network.forecast_timeseries_source: csv`, so forecast timeseries are read from the supplied CSV files instead of being generated synthetically (see 4).

2.8 Exploring the Results in a Notebook

With the notebook `notebooks/teaser_case_analysis.ipynb` you can reproduce Figures 9 to 18 of the ACER Paper on your own runs, and use it as a starting point for plots of your own. It plots the market outcomes, the flexibility needs and contributions, and the revenues of a few assets, from the first run of 2.1, `runs/teaser`. Your results can differ slightly from those in the paper, because the solver does not always return exactly the same solution.

A notebook is a document that mixes text, code and the plots the code makes. You open it with Jupyter, which runs in your web browser. The steps are the same on Windows, Linux and macOS:

1. **Run the model.** Run the first command of 2.1, from the root of the checkout. The notebook needs the finished run, `runs/teaser`.

2. **Install Jupyter.** Jupyter is not part of the `eu-flex` environment, so install it once:

   ```
   conda install -n eu-flex -c conda-forge jupyterlab
   ```

   Answer `y` when conda asks whether to proceed. This adds Jupyter to the environment and leaves the packages of the model as they are.

3. **Start Jupyter.** In a terminal (on Windows: the Miniforge Prompt), go to the root of the checkout, activate the environment and start JupyterLab:

   ```
   cd C:\eu-flex-model
   conda activate eu-flex
   jupyter lab
   ```

   On Linux and macOS, `cd` to wherever you downloaded the model instead. JupyterLab opens in your web browser. If it does not, copy the address starting with `http://localhost:8888/lab?token=` that it prints in the terminal into your browser. Leave the terminal open while you work with the notebook.

4. **Open the notebook.** In the file browser on the left of JupyterLab, double-click the `notebooks` folder, then `teaser_case_analysis.ipynb`. If JupyterLab asks you to select a kernel, choose *Python 3 (ipykernel)*.

5. **Run it.** From the menu, choose *Run* > *Run All Cells*. The plots appear below the cells that make them. The whole notebook takes less than a minute.

If you ran the model with the command of 2.1, the notebook finds the run without any changes. If you ran it in another working directory, set `FOLDER` in the cell under *Define folder and parameters* to that folder, then run all cells again. For an EVA run, the notebook plots the final iteration. In that cell, `EXPORT = 1` also saves the plotted data to an Excel file next to the notebook and to CSV files in the run folder.

JupyterLab saves your changes to the notebook file. To keep the original as it is, first save a copy under a new name with *File* > *Save Notebook As*. When you are done, choose *File* > *Shut Down* in JupyterLab, or press Ctrl+C twice in its terminal.

3 Configuration

3.1 How Configuration Works

You configure a run by editing one of several configuration files:

- `config/<my-config>.yaml`, passing the config file to snakemake with `--configfile`:

```
snakemake -c all --configfile config/<my-config>.yaml --directory
runs/<my-config>
```

  A copy of `config/teaser.yaml` is a good starting point.

- `src/euflex/eu_flex_model/config/config.default.yaml`, the complete default configuration with every option present and commented. Editing this changes the configuration for all runs. Note that this file is version controlled.

- `config/config.yaml` in the working directory, which is read on every run there (see 2.6).

The configuration is merged over the bundled defaults rather than replacing them, in this order, each winning over the one before:

1. The bundled `config.default.yaml`.

2. `config/config.yaml` in your working directory, if it exists.

3. Any file passed with `--configfile`.

4. Any value passed with `--config key=value`.

Because the merge is deep, a file passed at step 3 needs to contain only the values it changes. This is why `config/teaser.yaml` is short. A file like:

```yaml
eva:
  reserves:
    option: derated
```

changes that one value and leaves everything else under `eva.reserves` alone.

`config/config.yaml` is read from the working directory. Set `EUFLEX_CONFIG_DIR` to read it from somewhere other than `config/`.

You can keep as many configuration files as you like and pick one with `--configfile`. But every run writes to the same `resources/` and `results/` in the working directory, whichever file it was

started with. So running a second configuration in the same directory overwrites the first. To keep several runs, give each its own working directory (see 2.6 and 6).

Your configuration is validated against the schema bundled beside the defaults, `src/euflex/eu_flex_model/config/schema.yaml`, before the workflow starts. A typo in a key name, or a value out of range, fails immediately with a message naming the offending entry rather than failing hours into a run.

The rest of this section documents the options by topic. Defaults shown are the shipped defaults.

3.2 Run and Scenario

Which input networks, which years, and which intermediate files to keep.

Option	Default	Description
`run.name`	`""`	Name of the run. Also selects the pre-built set (`base_{name}`), so outside sensitivity mode it must match a directory under `data/pre-builts/`. The ACER Paper configurations set it to the name of their file, such as `teaser` and `teaser-eva`.
`run.parent`	`""`	Take pre-builts from a different run name than `run.name`. Useful for running several configurations off one set of inputs.
`run.keep_files`	`false`	Keep the large intermediate networks in `resources/networks/`. By default they are emptied once the next step has used them (see 4).
`scenario.ty`	`[2026, 2030]`	Target years to model. Each must exist in the input networks.
`scenario.cy`	`[2010, 2013]`	Climate years to model. Each must exist in the input networks.
`scenario.ft`	`[actual, forecast]`	Timeseries types. Leave as is unless you know why you are changing it.

The bidding zones and the time span of a run are whatever the input networks contain.

The model reads its input networks in one of two forms:

- **Pre-built networks**, single NetCDF files read from `data/pre-builts/base_{run}/input_scenarios/{ty}_{cy}.nc` in the checkout (or under `$EUFLEX_INPUT_DIR`, see 2.6), where `{run}` is `run.parent` if set and `run.name` otherwise. This release ships no pre-built networks.

- **Folders of network CSVs**, in sensitivity mode (`eva.sensitivity: true`), read from the folder named by `eva.build_network.sensitivity_path` (see 3.3 and 7.4). The ACER Paper model is supplied in this form.

A `ty` / `cy` combination that is not there fails at start-up with a missing-input error; list the directory to see which are available.

3.3 Network Construction

How the base network is assembled from the input networks.

Option	Default	Description
`eva.build_network.workflow_prebuilt.path`	`data/pre-builts/base_{run}/input_scenarios/{ty}_{cy}.nc`	Template for locating pre-builts. `{run}`, `{ty}` and `{cy}` are substituted.
`eva.sensitivity`	`false`	Build the base network from a folder of network CSVs instead of a pre-built network (see 7.4).
`eva.build_network.sensitivity_path`	`""`	Template for locating the folder of network CSVs in sensitivity mode, with `{run}`, `{ty}` and `{cy}` substituted, for example `data/teaser/ty_{ty}_cy_{cy}`. Here `{run}` is always `run.parent`, with no fallback to `run.name`. Relative paths as for `forecast_p_max_pu_path`. Left empty, it is the folder that `prepare_sensitivity` exports to (see 7.4).
`eva.build_network.forecast_timeseries_source`	`noise`	How day-ahead forecast timeseries are produced. `noise` perturbs the actual timeseries synthetically; `csv` reads them from files you supply.
`eva.build_network.forecast_p_max_pu_path`	`""`	Path to a generator availability CSV, used when the source is `csv`. A relative path is looked up in the working directory, then in the checkout.
`eva.build_network.forecast_p_set_path`	`""`	Path to a load CSV, used when the source is `csv`. Relative paths as for `forecast_p_max_pu_path`.
`eva.base_network.forecast_noise_std`	`1.0`	Multiplier on the standard deviation of synthetic forecast noise (load 3%, solar 8%, wind 12%), when the source is `noise`. Setting this to 0 makes the day-ahead forecast equal to the intraday actual.

Capacity and demand can be scaled at build time with the below options.

Option	Default	Description
`eva.build_network.scale_network.enable`	`true`	Enable capacity scaling. When false, `capacity_scaling_factor` is disabled; the load multipliers and minimum capacity still apply.
`eva.build_network.scale_network.capacity_scaling_factor`	`1`	Multiplier on generator capacities.
`eva.build_network.scale_network.load_scaling_factor`	`1`	Multiplier on demand.
`eva.build_network.scale_network.min_capacity`	`0.001`	Floor applied to generator capacity, in MW.
`eva.build_network.scale_network.da_res_ts_multiplier`	`1.0`	Multiplier on renewable availability in day-ahead networks, first iteration only.
`eva.build_network.scale_network.da_load_ts_multiplier`	`1.0`	Multiplier on demand in day-ahead networks, first iteration only.
`eva.build_network.scale_network.id_res_ts_multiplier`	`1.0`	As above, for intraday networks.
`eva.build_network.scale_network.id_load_ts_multiplier`	`1.0`	As above, for intraday networks.

3.4 Carriers and Unit Commitment

Which technologies exist, which are committable, and which the EVA loop is allowed to change.

Option	Default	Description
`eva.base_network.committable_carriers`	47 carriers	Carriers modelled with unit commitment: minimum stable generation, start-up costs, minimum up and down times.
`eva.base_network.id_free_commitment_carriers`	21 carriers	Gas, oil and hydrogen units and explicit DSR, allowed to start or stop in the intraday stage. Everything else is locked to its day-ahead commitment.
`eva.base_network.renewable_carriers`	solar, wind on/offshore, run-of-river, `renewables`	Carriers treated as renewable for curtailment and indicator purposes.
`eva.base_network.conventional_carriers`	46 carriers	Carriers treated as conventional.
`eva.base_network.decommissioned_carriers.Generator`	list	Generator carriers the EVA loop may only retire, never build. Only applied when the network has no `p_nom_decommissionable` column.

Option	Default	Description
`eva.base_network.decommissioned_carriers.StorageUnit`	list	Storage carriers the EVA loop may only retire. Only applied when the network has no `p_nom_decommissionable` column.
`eva.base_network.consider_p_nom_max`	`true`	Respect upper capacity limits when expanding.
`eva.base_network.pu_must_run`	`false`	Express must-run requirements per unit rather than in absolute MW.
`eva.base_network.consider_cyclic_soc`	`false`	Require storage to end the horizon at its starting state of charge.
`eva.base_network.remove_ramp_su_sd`	`false`	Drop start-up and shut-down ramp limits. Useful when they cascade into infeasibilities.

3.5 Market Interconnection

How cross-border exchange between bidding zones is represented.

Option	Default	Description
`eva.market_interconnection`	`physical`	`physical` uses the network's own lines and links. `ntc` applies net transfer capacities between zones; both give the same market model with the same results, but `ntc` adds region and link-type labels (core/nordic). `fbmc` applies flow-based market coupling with PTDFs and remaining available margins, which is more detailed and more expensive to solve.
`eva.eraa_year`	`2024`	Edition of the European Resource Adequacy Assessment (ERAA) to take the flow-based data from, used with `fbmc`. One of 2023 or 2024. The flow-based data is not included with this release (see below).

Flow-based market coupling with `fbmc` requires PTDFs and remaining available margins from the flow-based domains published by ENTSO-E with the ERAA. ENTSO-E publishes this data without a licence that allows redistribution, so this release does not include it, and `fbmc` fails until you supply it yourself: download the workbooks from the ERAA 2024 modelling data and place them in `data/` as `FB-Domain-CORE_Full_ERAA{year}.xlsx` and `FB-Domain-NORDIC_ERAA2024.xlsx`. The default `physical` setting and `ntc` need no extra files, and the ACER Paper model runs without flow-based market coupling.

3.6 Reserves

Balancing reserves: only Frequency Containment Reserve (FCR) and Frequency Restoration Reserve (FRR) are modelled.

Option	Default	Description
`eva.reserves.option`	`co-optimised`	`none` disables reserves entirely. `co-optimised` clears energy and reserves together in one problem. `derated` instead shrinks generator bounds by the reserve volumes before solving, which is much cheaper but does not let the market trade energy against reserve.
`eva.reserves.direction`	`[up, down]`	Which directions to model. Use `[up]` for upward only.

How reserves interact with the day-ahead and intraday markets is set per reserve type:

Option	Default	Description
`eva.reserves.fcr.id_blocked`	`1`	Fraction of the day-ahead FCR award that stays unavailable for intraday energy, from 0 to 1. By default FCR is fully blocked.
`eva.reserves.frr.id_blocked`	`0`	As above, for FRR, when `eva.reserves.frr.reserve_blocking_method` is `fraction`.
`eva.reserves.frr.reserve_blocking_method`	`imbalance`	How FRR blocking is computed. `fraction` blocks the static share `id_blocked`; `imbalance` blocks dynamically, based on the difference between the day-ahead and intraday forecasts. FCR always uses `fraction`.
`eva.reserves.fcr.penalty_cost`	`0.1`	Cost per MW of FCR in the objective, discouraging over-procurement. Small against energy prices, so it breaks ties without distorting the merit order.
`eva.reserves.frr.penalty_cost`	`0.1`	As above, for FRR.

Everything else about reserves is part of the network data rather than the configuration, so it is set in the pre-built network (or, in sensitivity mode, in its CSVs; see 7.4):

- **Requirements** per bidding zone, in MW, are the `reserve_FCR` and `reserve_FRR` columns of the buses.
- **Eligibility**: a generator or storage unit can provide a reserve type if its `reserve_FCR` or `reserve_FRR` column is true.

- **Provision limits**: the most a generator can offer, as a fraction of its capacity, is in its `max_fcr_provision`, `max_frr_provision_up` and `max_frr_provision_down` columns. Where these columns are absent, as in the ACER Paper model, the limit is 0.05 for every generator.
- **Minimum duration**, the hours a storage unit must be able to sustain its reserve, is the `reserve_FCR_min_duration` and `reserve_FRR_min_duration` columns of the buses. It sets the state of charge the unit must hold back.

3.7 Demand-Side Response

Two kinds of DSR are modelled. Explicit DSR is contracted load reduction, represented as generators. Implicit DSR is price-responsive load shifting, represented as storage.

Option	Default	Description
`eva.dsr.enabled`	`true`	Master switch for all DSR.
`eva.dsr.explicit_dsr.enabled`	`true`	Enable explicit DSR generators.
`eva.dsr.explicit_dsr.da_dispatch_fixed_fraction`	`0.0`	Fraction of the day-ahead DSR dispatch that must be held in intraday, from 0 to 1. Represents DSR that needs lead time and cannot be re-dispatched late.
`eva.dsr.implicit_dsr.enabled`	`true`	Enable implicit DSR storage units.
`eva.dsr.implicit_dsr.da_dispatch_fixed_fraction`	`0.0`	As above, for implicit DSR.

3.8 Storage Planning

Reservoir hydro and other long-duration storage cannot be scheduled sensibly inside a rolling horizon of a few days, because the horizon cannot see the value of water saved for next month. A coarse pre-optimisation over the whole time span of the network sets state of charge targets, which the market models then follow.

Option	Default	Description
`eva.storage_planning.enable`	`true`	Run the storage pre-optimisation.
`eva.storage_planning.po_resolution`	`8h`	Time resolution of the pre-optimisation, as an integer followed by `h`. Coarser is faster and less precise.
`eva.storage_planning.static_soc_target`	`0.5`	Fixed state of charge target used when planning is disabled.

Option	Default	Description
`eva.storage_planning.storage_plan_frequency`	`5`	Re-plan every N EVA iterations.
`eva.storage_planning.storage_constraints_enabled`	`true`	Apply the network's own reservoir state-of-charge limits in the pre-optimisation and market models. Storage plan targets always apply.
`eva.storage_planning.preopt_targeted_carriers`	3 hydro carriers	Which storage carriers the pre-optimisation plans.
`eva.storage_planning.use_cyclic_soc`	`true`	Require the planned trajectory to end where it started.

Set `po_resolution` no coarser than the time span of your input networks. The ACER Paper configurations use `4h`.

3.9 Dummy Storage Optimisation

A counterfactual study. The model adds hypothetical "dummy" storage of a given duration and asks how much would be needed to hit a curtailment target. Running it across several durations and targets produces the storage-versus-curtailment trade-off curve.

Option	Default	Description
`eva.dummy_storage_optimization.enable`	`true`	Run the study. It multiplies runtime by the number of `mh` x `ct` combinations, so switch it off if you only want the base case.
`eva.dummy_storage_optimization.energy_power_ratios`	`[1, 2, 4, 6, 12, 24, 48, 168]`	Storage durations in hours. Each becomes an `mh` wildcard.
`eva.dummy_storage_optimization.curtailment_targets`	`[0.05]`	Maximum share of available renewable energy that may be curtailed. Each becomes a `ct` wildcard.
`eva.dummy_storage_optimization.curtailment_targets_per_bz`	`false`	Read the targets per bidding zone from `data/ds_curtailment_targets_by_bz.csv`, keyed `ct1` to `ctN`, instead of applying the values above to every zone.
`eva.dummy_storage_optimization.shared_across_countries`	`true`	Re-optimise cross-border link flows together with the dummy storage. When false, the flows are fixed to the intraday market result.
`eva.dummy_storage_optimization.minimize_system_costs`	`true`	Minimise total system cost. When false, minimises dummy storage capacity alone.

Option	Default	Description
`eva.dummy_storage_optimization.network_needs`	`false`	Account for randomly generated mock network needs (up to 80% of renewable availability) in the dummy storage optimisation; these are placeholder data.
`eva.dummy_storage_optimization.enable_load_spillage`	`true`	Allow load spillage generators, which keep the problem feasible at aggressive targets.
`eva.dummy_storage_optimization.load_spillage_penalty`	`10000.0`	Penalty in EUR/MWh on load spillage when `minimize_system_costs` is false, so that capacity minimisation does not spill load instead of building dummy storage.
`eva.dummy_storage_optimization.export_models`	`false`	Write the `.lp` problem files for inspection.
`eva.dummy_storage_optimization.conventional_floor_carriers`	`null`	Conventional carriers whose technical minimum enters the lower bound on conventional output in the dummy storage optimisation. `null` selects all conventional carriers, `[]` none; the must-run requirement always applies.
`eva.dummy_storage_optimization.solver_options.highs` / `.gurobi`	`highs-lp-baseline` / `gurobi-optimal-dummy-storage`	Option set used for the dummy storage optimisation, per solver, in place of `solving.solver.options` (see 3.17).

With eight ratios and one target this is eight extra optimisations per year combination, which typically dominates the runtime of a small case.

3.10 Forced Outages

Unplanned generator failures, sampled stochastically. They apply to the intraday (actual) market only, since a forecast cannot anticipate them.

Option	Default	Description
`eva.forced_outages.enable`	`true`	Sample forced outages.
`eva.forced_outages.samples`	`1`	Number of Monte Carlo samples. Each becomes an `fo` wildcard and multiplies the number of intraday simulations.
`eva.forced_outages.seed`	`[18]`	Random seeds, one per sample. The list length must match `samples`. Fixing them makes runs reproducible.

Option	Default	Description
`eva.forced_outages.n_units_per_fleet`	`100`	Individual units modelled per aggregated fleet. Higher values give smoother availability profiles and a more realistic partial-outage distribution. Only applied when the network has no `n_units` column.

3.11 Maintenance Planning

Planned outages, scheduled to avoid tight periods. This is solved with simulated annealing rather than as a linear program.

Option	Default	Description
`solving.maintenance_planning.enable`	`true`	Plan maintenance.
`solving.maintenance_planning.source`	`optimize`	`optimize` runs the simulated annealing. `plan-id` reads a previously computed plan from `csv_path`.
`solving.maintenance_planning.csv_path`	`""`	Plan to read when the source is `plan-id`.
`solving.maintenance_planning.min_duration_hours`	`24`	Shortest maintenance window. Only applied when the network has no `planned_outage_min_hours` column.
`solving.maintenance_planning.annual_windows`	`1`	Maintenance windows per unit per year. Only applied when the network has no `planned_outage_annual_windows` column.
`solving.maintenance_planning.penalty_weight`	`1000.0`	Weight on the normalised outage term; the recommended range is 1 to 1000.
`solving.maintenance_planning.sa_max_iterations`	`5000`	Simulated annealing iterations. The main quality-versus-time lever.
`solving.maintenance_planning.sa_initial_acceptance`	`0.8`	Probability of accepting the median worsening move at the start. The starting temperature is calibrated per bus from sampled moves to match it.
`solving.maintenance_planning.sa_alpha`	`0.997`	Cooling rate per iteration.
`solving.maintenance_planning.sa_units_per_move`	`3`	Units perturbed per candidate move.
`solving.maintenance_planning.sa_parallel_runs`	`1`	Independent simulated annealing runs; the best result wins.
`solving.maintenance_planning.sa_random_seed`	`4`	Seed, for reproducibility.

Option	Default	Description
`eva.maintenance_plan_frequency`	`5`	Re-plan every N EVA iterations.

Maintenance planning is skipped automatically when the modelled period is shorter than a year, since there is nothing to schedule around.

3.12 Price Caps

Market price caps follow the harmonised maximum clearing price methodology: when prices hit the cap often enough, the cap rises.

Option	Default	Description
`eva.price_cap.enable`	`true`	Apply and update price caps.
`eva.price_cap.day_ahead_values`	`[5000, 5000, 5000, 5000]`	Starting day-ahead caps in EUR/MWh, one per target year. Needs at least as many entries as `scenario.ty` .
`eva.price_cap.intraday_values`	`[10000, 10000, 10000, 10000]`	Starting intraday caps in EUR/MWh, one per target year.
`eva.price_cap.maximum`	`20000`	Ceiling the cap can never exceed, in EUR/MWh.
`eva.price_cap.multi_year_method`	`expected`	How caps carry across target years. `expected` uses the mean final cap from the previous target year across climate years; `interpolation` , `last-year` and `none` are also available.

3.13 Negative Pricing

Renewables with subsidy or contractual structures may bid below zero rather than curtail. This is modelled by splitting a share of each renewable's capacity into a companion generator with a negative marginal cost.

Option	Default	Description
`eva.base_network.negative_pricing.enable`	`false`	Enable negative bidding.
`eva.base_network.negative_pricing.share`	`0.3`	Default share of capacity that bids negative, from 0 to 1.
`eva.base_network.negative_pricing.value`	`-50.0`	Default bid in EUR/MWh. Must be zero or below.

Option	Default	Description
`eva.base_network.negative_pricing.carriers`	per-carrier map	Overrides `share` and `value` for individual carriers.

Negative pricing applies only to four fixed carriers: `Solar (PV)`, `Wind Onshore`, `Wind Offshore` and `Hydro - Run of River (Turbine)`. The default `carriers` map sets values for all four, so the global `share` and `value` only apply to a carrier removed from the map. Adding other carriers to the map has no effect.

```yaml
eva:
  base_network:
    negative_pricing:
      enable: true
      carriers:
        Solar (PV):
          share: 0.4
          value: -30.0
```

3.14 The EVA Loop

How capacities evolve between iterations, and when the loop stops.

Option	Default	Description
`eva.enable`	`true`	Run the capacity update. When false, capacities stay fixed and the model is a pure market simulation.
`eva.max_it`	`3`	Maximum iterations. The loop also stops early once capacities converge, i.e. once no asset changes capacity.
`eva.capacity_update.increment`	`800`	Capacity added or removed per asset per iteration, in MW.
`eva.capacity_update.threshold`	`70000`	NPV per MW above which an asset is considered viable and expanded, or below minus it an asset is considered nonviable and decommissioned. Assets within ±threshold count as converged.
`eva.capacity_update.linear_decay`	`true`	Shrink the increment over successive iterations, which helps the loop settle instead of oscillating.
`eva.capacity_update.linear_decay_iterations`	`20`	Iterations over which the increment decays. Defaults to `max_it` when null.

Option	Default	Description
`eva.capacity_update.linear_decay_min_step`	`10`	Smallest increment the decay will reach, in MW.
`eva.enable_mothballing`	`true`	Allow units to go temporarily offline and return later, instead of retiring permanently.
`eva.enable_lifetime_extension`	`true`	Allow units to run past their nominal lifetime when it pays. The decision is revisited in later iterations.

3.15 Costs and Net Present Value

Option	Default	Description
`costs.discount_rate`	`0.05`	Discount rate used when a technology-specific hurdle rate is unavailable.
`costs.use_hurdle_rates`	`true`	Use technology-specific hurdle rates where available, rather than the single discount rate.
`costs.exogenous_revenues.enable`	`false`	Add revenue streams earned outside the modelled market, such as heat or capacity payments. See below.
`costs.necp_targets.enable`	`false`	Compare renewable generation against National Energy and Climate Plan targets. Needs a targets file; see below.
`costs.necp_targets.file`	`data/in_necp_targets.csv`	NECP targets file. Relative paths are resolved against the checkout.
`costs.co2_targets.enable`	`false`	Compare emissions against member state policy targets. Needs a targets file; see below.
`costs.co2_targets.file`	`data/in_necp_targets.csv`	CO2 targets file. Relative paths are resolved against the checkout.
`eva.npv.weighted_calculation.enabled`	`false`	Weight climate years unequally in the NPV. When false, all are weighted alike.
`eva.npv.weighted_calculation.cy_weights`	`{2013: 0.5, 2010: 0.5}`	Per-climate-year weights. Must sum to 1 when weighting is enabled.

Exogenous revenues are read from per-asset columns on the generators and storage units, in EUR/kW/year: `CHP_revenue`, `balancing_revenue`, `CM_revenue` and `congestion_revenue`. The ACER Paper model does not carry these columns, so enabling the option changes nothing unless

you add them yourself to `generators.csv` or `storage_units.csv` in sensitivity mode (7.4). The model does not read the `costs.exogenous_revenues.file` key in the defaults.

The NECP and CO2 targets file is not shipped with the model. To use these comparisons, supply your own CSV at the configured path. If the file is missing, the run logs an error and carries on, with the targets left as NaN. The CSV needs the columns `year`, `country_short`, `category`, `kpi`, `sector`, `variable1`, `variable2`, `variable3`, `value` and `unit`:

- Renewable generation targets are the rows with `kpi` = `Electricity generation`, `variable1` = `Renewable` and `variable3` = `Total`, one per `variable2` (`Wind`, `Solar`, `Hydro`, `Bio`, `Other Renewable`).
- Emission targets are the rows with `category` = `Emissions`, `kpi` = `GHG emissions`, `sector` = `Total` and `variable1` = `Total`.

3.16 Flexibility Indicators

The indicators are the main point of the Economic Viability Assessment. Most are computed unconditionally from the final iteration; these options tune the ones that have choices.

Option	Default	Description
`eva.short_term_requirements.method`	`simulation`	How short-term flexibility need is derived. `simulation` takes the observed day-ahead to intraday residual load difference. `probabilistic` fits forecast error distributions per system condition instead.
`eva.short_term_requirements.condition_1_months`	`[11, 12, 1, 2, 3]`	Months making up system condition 1, conventionally winter. Only used with `method: probabilistic`.
`eva.short_term_requirements.condition_2_months`	`[4, 5, 6, 7, 8, 9, 10]`	Months making up system condition 2, conventionally summer. Only used with `method: probabilistic`.
`eva.short_term_requirements.condition_a_hours`	`[7..21]`	Hours making up condition a, conventionally day. Only used with `method: probabilistic`.
`eva.short_term_requirements.condition_b_hours`	`[22..6]`	Hours making up condition b, conventionally night. Only used with `method: probabilistic`.
`eva.short_term_requirements.percentile_low`	`0.1`	Lower percentile of the forecast error distribution. Only used with `method: probabilistic`.
`eva.short_term_requirements.percentile_high`	`99.9`	Upper percentile of the forecast error distribution. Only used with `method: probabilistic`.

Option	Default	Description
`eva.ramping_shortfall.enable`	`true`	Compute how much curtailment and unserved energy is caused by ramping limits, by re-running the final intraday case without them.
`eva.ramping_contribution.n_worst_events`	`[1, 10, 25]`	Compute each technology's share of the N steepest ramping events, for each N listed.

3.17 Market Clearing and Solver

The modelled markets are cleared over a rolling horizon: a window of `horizon` hours is solved, the first part is kept, and the window advances with an `overlap` of hours re-solved to smooth the seam.

Option	Default	Description
`solving.options.horizon`	`72`	Window length in hours.
`solving.options.overlap`	`24`	Hours of overlap between consecutive windows.
`solving.options.linearized_unit_commitment`	`true`	Relax the binary on/off decisions into a linear program. Dramatically faster, at some loss of fidelity.
`solving.options.use_convex_hull_tightening`	`false`	Tighten the relaxation so it sits closer to the integer solution.
`solving.options.load_shedding`	`true`	Allow demand to go unserved at a high penalty cost. Keep this on: it converts infeasibility into a priced, visible result.
`solving.options.load_spill`	`true`	Allow surplus generation to be spilled.
`solving.options.reserve_shedding`	`false`	Allow reserve requirements to go unmet at a penalty.

Solver selection:

Option	Default	Description
`solving.solver.name`	`highs`	One of `highs`, `gurobi`, `xpress`, `cbc`, `cplex`, `knitro`. Only `highs` and `gurobi` have bundled option sets and are tested; the others need option sets of your own.
`solving.solver.options`	`highs-default`	Which named option set under `solving.solver_options` to use.

Option	Default	Description
`solving.solver_options.<name>`	see config	Solver-specific settings, passed through verbatim.

```yaml
solving:
  solver:
    name: gurobi
    options: gurobi-default
```

Bundled option sets include `highs-default`, `highs-lp-baseline`, `highs-hipo-lp-baseline`, `gurobi-default`, `gurobi-optimal`, `gurobi-milp-baseline` and `gurobi-optimal-dummy-storage`. To add your own, define a new key under `solving.solver_options` and name it in `solving.solver.options`.

3.18 Temporal Clustering

On long runs, most weeks resemble each other. Clustering solves a representative subset each iteration and only occasionally does the full year, which cuts runtime substantially at some cost in accuracy.

Option	Default	Description
`eva.advanced_clustering.enable`	`false`	Enable clustering.
`eva.advanced_clustering.n_clusters`	`12`	Number of representative chunks per iteration.
`eva.advanced_clustering.horizon_days`	`7`	Chunk length in days.
`eva.advanced_clustering.full_run_frequency`	`10`	Run every chunk once every N iterations, to refresh the clusters.
`eva.advanced_clustering.random_seed`	`42`	Seed for the k-medoids clustering.

Not every iteration is clustered. A full run — every chunk, not just the representative ones — is forced on the first iteration, on every `full_run_frequency`-th iteration, on the last iteration, and on the iteration immediately after one that converged on clustered results, so that a convergence reached on a subset is always confirmed against the full year.

3.19 Perfect Foresight and Infeasibility Handling

Option	Default	Description
`eva.perfect_foresight.enable`	`false`	Solve one market on actual data instead of separate day-ahead and intraday stages. Gives an optimistic upper bound, since nothing is ever forecast wrong.
`eva.infeasibility_threshold`	`0.0`	Fraction of rolling-horizon windows allowed to be infeasible before the run is considered failed, from 0 to 1.
`eva.fail_on_infeasibility`	`true`	Stop the workflow when the threshold is exceeded. Set false to push through and inspect the damage.
`eva.patching_tol_status`	`1e-4`	Tolerance for rounding near-integer commitment values coming out of the relaxed problem.
`eva.patching_tol_ramp`	`1e-4`	Largest silent ramp correction allowed when reconciling intraday with day-ahead. Bigger corrections warn and are skipped.
`eva.export_lp_models`	`false`	Write every market problem to `.lp` for inspection. Produces a lot of data.

3.20 Outputs and Logging

Option	Default	Description
`logging.level`	`INFO`	One of `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`.
`plotting.enable`	`false`	Generate summary figures and the plot catalogue.
`plotting.tech_colors`	colour map	Colour per carrier in the figures.
`plotting.nice_names`	name map	Display names per carrier.
`eva.shadow_prices.enable`	`false`	Extract dual variables to `shadow_prices.nc`.
`eva.shadow_prices.constraints`	4 constraints	Which constraints to extract, named as `Component-constraint`, for example `Bus-nodal_balance`.
`eva.soc_validation_zones`	`[...]`	Zones for which storage validation plots are produced.

Option	Default	Description
`eva.validation_plot_frequency`	`0`	Produce validation plots for the final iteration of a full (unclustered) run when `final_it % N == 1`. `0` disables them.
`eva.integration_tests.enable`	`false`	Run consistency checks on market results during the workflow.
`eva.integration_tests.fail_on_error`	`false`	Fail the run when a check is violated, rather than just reporting.

3.21 Environment Variables

A few settings concern where the model finds and writes files, or how it shares a solver licence, rather than what it models. None of them is needed to run the ACER Paper model. They are set as environment variables, not in the configuration file: `export NAME=value` in a Linux or macOS shell, `set NAME=value` in the Miniforge Prompt, `$env:NAME = "value"` in PowerShell, in the terminal you then run `snakemake` from.

Variable	Default	Description
`EUFLEX_INPUT_DIR`	the checkout, found by searching upwards from the working directory	Checkout holding `data/`, including the prebuilt networks. Needed when running from outside the checkout (see 2.6).
`EUFLEX_CONFIG_DIR`	`config`	Directory holding your `config.yaml`.
`EUFLEX_SENSITIVITY_DIR`	`var/workflow/sensitivity`	Where `prepare_sensitivity` exports the network CSVs, and where sensitivity mode reads them when `eva.build_network.sensitivity_path` is empty (see 7.4).
`EUFLEX_WORKING_DIR`	`var`	Directory for per-run scratch data, such as infeasibility exports.
`EUFLEX_GUROBI_LOCKING`	`false`	Set to `true` to let only one job at a time use Gurobi, for single-use licences such as Gurobi WLS that would otherwise be claimed by several parallel jobs at once.
`EUFLEX_GUROBI_LOCK_PATH`	`/var/lock/euflex/gurobi_wls.lock`	Lock file used when `EUFLEX_GUROBI_LOCKING` is `true`. Its directory must exist and be writable.

The relative defaults for `EUFLEX_SENSITIVITY_DIR` and `EUFLEX_WORKING_DIR` are relative to the checkout when the model can find it, and to the working directory otherwise. `EUFLEX_CONFIG_DIR` is always relative to the working directory.

4 Intermediate Files

Everything under `resources/` in your working directory is intermediate and safe to delete once a run is finished.

```
resources/
├── capacities/          # installed capacity and unit states per
asset per iteration
├── cleanup/             # markers recording which intermediate
networks have been emptied
├── convergence/         # whether EVA converged
├── dispatch/            # dispatch timeseries for every market model
├── market_models/       # .lp formulations and per-window solver
status
├── networks/            # intermediate PyPSA networks
├── npv/                 # NPV results and computation logs
├── outages/             # sampled generator and link outage profiles
├── plots/               # process validation plots
├── pre_optimization/    # storage plans and maintenance plans
└── price_caps/          # price cap timeseries
```

`capacities/unit_states_{it}.csv` holds one row per asset and target year with the state of the asset after iteration `it` (for example `active`, `mothballed` or `capacity_removed`), its capacity and its NPV before and after the lifetime-extension adjustment.

The networks in `resources/networks/` are the useful ones for debugging:

- `base_network_{ty}_{cy}_{ft}_{it}.nc` is the starting point of an iteration: the pre-built plus your configuration in iteration 0, and the previous iteration's network with its capacity updates after that. In iteration 0, the day-ahead forecast timeseries, synthetic noise or CSV, are applied here for `ft=forecast`; later iterations carry them over.
- `resampled_network_{ty}_{cy}_forecast_{it}.nc` is the same network at the coarse resolution used for storage planning. It is only written in iterations where the storage plan is recomputed (see `storage_plan_frequency` in 3.8).
- `day_ahead_network_before_cap_update_{ty}_{cy}_forecast_{it}.nc` and `intraday_network_before_cap_update_{ty}_{cy}_actual_{fo}_{it}.nc` hold the market results as solved, before the price caps are updated (see 3.12).
- `day_ahead_network_{ty}_{cy}_forecast_{it}.nc` and `intraday_network_{ty}_{cy}_actual_{fo}_{it}.nc` hold the same results with the updated price caps applied. The NPV calculation reads these.

- `day_ahead_network_integer_{ty}_{cy}_forecast_{it}.nc` holds the day-ahead result with integer unit commitment. It is only filled when `solving.options.linearized_unit_commitment` is `false`, and is an empty network otherwise.
- `intraday_network_without_ramping_{ty}_{cy}_actual_{fo}_{it}.nc` is the counterfactual without ramping limits, used for the ramping shortfall indicator. It is only computed for the final iteration, and only with `eva.ramping_shortfall.enable`, which the ACER Paper configurations turn off.

To save disk space, most of these files are emptied to zero bytes once the following steps have used them: the `before_cap_update` networks at the end of each iteration, and most of the rest at the end of the next one. What is left is iteration 0's base networks and the final iteration's networks. The empty files keep their timestamps, so Snakemake does not try to rebuild them. To keep everything for inspection, set `run.keep_files: true`, as all ACER Paper configurations do.

5 Model Outputs

The main output is a single NetCDF file, `results/assessment/model_outputs.nc`, holding every flexibility indicator, market output and EVA result. Per-combination files `model_outputs_{ty}_{cy}.nc` are written alongside it.

```
results/
├── assessment/
│   ├── dummy_storage_optimization/    # networks from the dummy
storage study
│   ├── plots/                         # figures, if plotting.enable
is true
│   ├── model_outputs.nc               # all results
│   ├── model_outputs_{ty}_{cy}.nc     # results for one combination
│   └── shadow_prices.nc               # dual values, if
eva.shadow_prices.enable is true
└── networks/                          # final market networks
    ├── day_ahead_network_{ty}_{cy}.nc
    ├── intraday_network_{ty}_{cy}.nc
    └── intraday_network_without_ramping_{ty}_{cy}.nc
```

5.1 File Structure

The dataset is an xarray DataTree with two groups:

- **`/base`**: the main EVA results, around 70 variables.

- **`/dummy_storage_optimization`**: the dummy storage study, around 40 variables, with two extra dimensions for comparing configurations.

Dimensions:

Dimension	Group	Description
`TY`	both	Target year
`CY`	both	Climate year
`carrier`	both	Technology
`BZ`, `bidding_zone`	both	Bidding zone
`hour`, `day`, `week`	both	Time indices
`condition`	base	System condition, for example `1a` for winter day. Present only when `short_term_requirements.method` is `probabilistic`
`interconnection`	both	Border between two zones
`iteration`	base	EVA iteration
`asset`	base	Individual asset name
`Generator`, `StorageUnit`, `Load`	base	Component names
`generator`, `storage_unit`	base	Component names, in the reserve and reservoir variables
`mh`	dummy storage	Storage duration in hours
`ct`	dummy storage	Curtailment target scenario

The dummy storage group is computed from the dummy storage networks only and does not carry the same variables as the base: it has no day-ahead, short-term need, reserve, reservoir or EVA variables. It aggregates `carrier` into nine groups (`renewables`, `conventionals`, `storage`, `Dummy Storage`, `AC`, `Interconnectors`, `load`, `load shedding`, `load spillage`) rather than the base group's full technology breakdown.

The `TY` and `CY` coordinates are strings (`"2030"`, not `2030`), as are `mh` and `ct`. Select with `sel(TY="2030", CY="2013")`.

The base group also holds `_color` and `_nice_name`, per-carrier plotting colours and display names taken from `plotting.tech_colors` and `plotting.nice_names`.

5.2 Variable Groups

Variables carry `description`, `units` and `kind` attributes. `kind` places them in one of the categories below. Some variables only appear with certain settings, as noted.

Flexibility indicators (`kind="flexibility_indicator"`)

Variable	Dimensions	Units	Description
`curtailment`	`(TY, CY, hour, carrier, BZ)`	MWh	Renewable generation curtailed
`curtailment_aggregated`	`(TY, CY, carrier, BZ)`	MWh	Total curtailment by carrier and zone
`residual_load`	`(TY, CY, BZ, hour)`	MWh	Demand minus potential renewable and must-run supply
`daily_flexibility_need`	`(TY, CY, BZ, day)`	MWh	Sum of absolute deviations from the daily mean residual load
`daily_flexibility_need_aggregated`	`(TY, CY, BZ)`	MWh	Sum over all days
`weekly_flexibility_need`	`(TY, CY, BZ, week)`	MWh	Deviation of each day's mean residual load from the weekly mean, summed over the hours of each day
`weekly_flexibility_need_aggregated`	`(TY, CY, BZ)`	MWh	Sum over all weeks
`seasonal_flexibility_need`	`(TY, CY, BZ)`	MWh	Deviation of each month's mean from the annual mean, summed over the hours of each month
`maximum_ramping_requirement`	`(TY, CY, BZ)`	MWh	Largest hour-to-hour change in residual load
`ramping_shortfall_upward`	`(TY, CY, hour, BZ)`	MWh	Extra curtailment caused by ramping limits
`ramping_shortfall_upward_aggregated`	`(TY, CY, BZ)`	MWh	Signed sum over all hours
`ramping_shortfall_downward`	`(TY, CY, hour, BZ)`	MWh	Extra unserved energy caused by ramping limits
`ramping_shortfall_downward_aggregated`	`(TY, CY, BZ)`	MWh	Signed sum over all hours
`technology_contributions_daily`	`(TY, CY, carrier, BZ)`	MWh	Each technology's contribution to daily flexibility
`technology_contributions_weekly`	`(TY, CY, carrier, BZ)`	MWh	Same, weekly

Variable	Dimensions	Units	Description
technology_contributions_seasonal	(TY, CY, carrier, BZ)	MWh	Same, seasonal
ramping_contribution_n1 / _n10 / _n25	(TY, CY, BZ, carrier)	fraction	Each technology's share of the worst 1, 10 or 25 ramping events
cross_border_contributions_daily / _weekly / _seasonal	(TY, CY, BZ)	MWh	Cross-border flow contribution to daily, weekly and seasonal flexibility. Only when the network has links

Short-term flexibility needs come from one of two sets of variables, depending on eva.short_term_requirements.method (see 3.16). With simulation, the default:

Variable	Dimensions	Units	Description
short_term_need_upward_hourly / _downward_hourly	(TY, CY, BZ, hour)	MWh	Short-term need: the intraday minus day-ahead residual load, by direction
total_short_term_need_upward / _downward	(TY, CY, BZ)	MWh	Sum over all hours
short_term_need_hourly	(TY, CY, BZ, hour)	MWh	Signed short-term need
short_term_need_upward_p99_9 / _upward_p99 / _downward_p1 / _downward_p0_1	(TY, CY, BZ)	MWh	99.9th, 99th, 1st, 0.1th percentile of the signed hourly short-term need
aggregated_short_term_need_upward_p99_9 / _upward_p99 / _downward_p1 / _downward_p0_1	(TY, BZ)	MWh	Same, pooled across climate years
short_term_shortfall_curtailment	(TY, CY, hour, BZ)	MWh	Intraday curtailment in excess of day-ahead
short_term_shortfall_curtailment_total / _max	(TY, CY, BZ)	MWh	Sum and maximum over all hours
short_term_shortfall_ens	(TY, CY, BZ, hour)	MWh	Intraday energy not served in excess of day-ahead
short_term_shortfall_ens_total / _max	(TY, CY, BZ)	MWh	Sum and maximum over all hours
short_term_technology_contributions_hourly	(TY, CY, carrier, BZ)	MWh	Each technology's contribution to hourly short-term needs
short_term_cross_border_contributions_hourly	(TY, CY, BZ)	MWh	Cross-border contribution to hourly short-term needs. Only when the network has links

With probabilistic:

Variable	Dimensions	Units	Description
`forecast_error_p_low` / `_p_high`	`(TY, CY, condition, BZ)`	MWh	Forecast error percentiles per system condition
`residual_flex_capacity_up` / `_down`	`(TY, CY, hour, BZ)`	MWh	Flexible capacity left after reserves
`probabilistic_uncovered_need_up` / `_down`	`(TY, CY, hour, BZ)`	MWh	Need not covered by available flexibility
`probabilistic_expected_ens` / `_expected_curtailment`	`(TY, CY, hour, BZ)`	MWh	Expected energy not served and curtailment from uncovered need

Market outputs (`kind="market_output"`)

Variable	Dimensions	Units	Description
`supply` / `withdrawal`	`(TY, CY, hour, carrier, BZ)`	MWh	Energy supplied and withdrawn
`market_prices_day_ahead`	`(TY, CY, hour, bidding_zone)`	€/MWh	Day-ahead clearing prices
`market_prices_intraday`	`(TY, CY, hour, bidding_zone)`	€/MWh	Intraday clearing prices
`average_prices`	`(TY, CY, bidding_zone)`	€/MWh	Mean intraday price
`price_volatility`	`(TY, CY, bidding_zone)`	€/MWh	Standard deviation of intraday price
`zero_negative_price_hours`	`(TY, CY, bidding_zone)`	hours	Count of hours at or below zero
`total_generation_day_ahead` / `_intraday`	`(TY, CY, day, carrier, BZ)`	MWh	Daily generation by technology
`total_co2_emissions_day_ahead` / `_intraday`	`(TY, CY, bidding_zone)`	t CO2	Emissions
`net_power_transfer_day_ahead` / `_intraday`	`(TY, CY, hour, interconnection)`	MWh	Net flow between zones
`reserve_capacity_day_ahead_r_FCR_up` / `_r_FCR_down` / `_r_FRR_up` / `_r_FRR_down`	`(TY, CY, hour, generator)`	MW	Day-ahead reserve awarded per unit
`reservoir_storage_levels`	`(TY, CY, hour, storage_unit)`	MWh	Planned reservoir trajectory
`demand`	`(TY, CY, hour, Load)`	MW	Electrical demand
`storage_units_charging`	`(TY, CY, hour, StorageUnit)`	MW	Storage charging

Variable	Dimensions	Units	Description
`load_spillage`	`(TY, CY, hour, Generator)`	MW	Spilled surplus

EVA outputs (`kind="eva_output"`)

Variable	Dimensions	Units	Description
`production_capacities`	`(TY, CY, carrier, BZ)`	MW	Installed generation capacity
`storage_capacities`	`(TY, CY, carrier, BZ)`	MWh	Installed storage capacity
`commissioned_capacity`	`(TY, iteration, carrier, BZ)`	MW	Capacity added per iteration
`decommissioned_capacity`	`(TY, iteration, carrier, BZ)`	MW	Capacity retired per iteration
`mothballed_capacity`	`(TY, iteration, carrier, BZ)`	MW	Capacity mothballed (temporarily taken offline) per iteration
`npv_per_mw`	`(TY, iteration, asset)`	EUR/MW	Net present value per asset, with coordinates `asset_carrier` and `asset_BZ`

Adequacy indicators (`kind="adequacy_indicator"`)

Variable	Dimensions	Units	Description
`energy_not_served`	`(TY, CY, BZ, hour)`	MWh	Unserved demand
`total_energy_not_served`	`(TY, CY, BZ)`	MWh	Annual total
`loss_of_load`	`(TY, CY, BZ)`	hours	Hours with unserved demand

Policy target comparisons (`kind="necp_comparison"` and `kind="co2_comparison"`)

With `costs.necp_targets.enable`, `necp_res_generation_model`, `necp_res_generation_target` and `necp_res_generation_diff_pct` compare modelled renewable generation with the NECP targets, in TWh and percent. With `costs.co2_targets.enable`, `co2_emissions_model`, `co2_emissions_target` and `co2_emissions_diff_pct` do the same for emissions, in Mt CO2. Both need a targets file; see 3.15.

5.3 Accessing the Data

```
import xarray as xr

dt = xr.open_datatree("runs/teaser/results/assessment/
```

```
model_outputs.nc")

# What is in there
print(dt)

# The main results group
base = dt["base"].ds
print(list(base.data_vars))

# One variable, for one year and zone
base["residual_load"].sel(TY="2030", CY="2013", BZ="N1")

# Variable metadata
print(base["residual_load"].attrs)

# Total curtailment across carriers and zones
base["curtailment"].sum(dim=["carrier", "BZ"])

# Compare dummy storage configurations, if the dummy storage
optimisation is enabled
ds = dt["dummy_storage_optimization"].ds
ds["production_capacities"].sel(mh="8", ct="ct2", carrier="Dummy
Storage")
```

Plotting a single zone:

```
import matplotlib.pyplot as plt

base["residual_load"].sel(TY="2030", CY="2013", BZ="N1").plot()
plt.show()
```

Shadow prices land in their own file when `eva.shadow_prices.enable` is `true` (see 3.20):

```
sp = xr.open_dataset("runs/teaser/results/assessment/
shadow_prices.nc")
print(list(sp.data_vars))

# Nodal prices are the dual of the nodal balance constraint.
# Names follow {network}_mu_{constraint}, where network is da or id.
sp["da_mu_nodal_balance"].sel(TY="2030", CY="2013")
```

6 Output Directory Structure

Almost everything a run produces is written relative to the working directory: the folder you pass to `--directory`, such as `runs/teaser`, or the folder you run `snakemake` from without it (see 2.6).

```
<working directory>/
├── config/
│   └── config.yaml        # optional: configuration read on every run
├── benchmarks/            # per-rule runtime and memory, written by
Snakemake
├── logs/                  # one log per rule, plus the Snakemake log
├── resources/             # intermediate files (see section 4)
└── results/               # final outputs (see section 5)
```

Snakemake also keeps its own bookkeeping in `.snakemake/`, and sensitivity mode leaves a `sensitivity_export_<run.name>.flag` marker in the working directory (see 7.4). Two kinds of output go to the checkout (the model's root directory) instead of the working directory: per-run scratch data, such as infeasibility exports, goes to `<checkout>/var/<run.name>/`, and sensitivity exports go to `<checkout>/var/workflow/sensitivity/` (see 7.4). Both locations can be changed with environment variables (see 3.21).

`config.default.yaml` and `schema.yaml` are not copied here: they stay inside the installed package, at `src/euflex/eu_flex_model/config/`.

There is no per-run subdirectory under the working directory: the working directory *is* the run directory, and a second run in the same one overwrites the first. `run.name` does not change this — it selects which pre-built network set to read (see 3.2), not where output goes, although runs with the same `run.name` share the scratch folder under `<checkout>/var/`. To keep runs apart, give each its own working directory with `--directory`, which is also how you run two configurations side by side against the same installation: each gets its own `resources/`, `results/`, and `logs/`, all reading the same bundled `config.default.yaml`.

7 Worked Examples

All examples start from the ACER Paper model. Each is a configuration snippet that changes only what it names: save it as a file in `config/`, and pass it after `config/teaser.yaml`, which it then overrides (see 3.1). Give each example its own working directory, so that it does not overwrite the results of another run.

7.1 Summary Plots

Save this as `config/plots.yaml`:

```yaml
plotting:
  enable: true
```

and pass it after the ACER Paper configuration:

```
snakemake -c all --configfile config/teaser.yaml config/plots.yaml --
directory runs/teaser-plots
```

The figures land in `runs/teaser-plots/results/assessment/plots/`.

7.2 With the Dummy Storage Optimisation

`config/teaser.yaml` leaves the dummy storage optimisation off; the dummy storage runs of the ACER Paper have configurations of their own (see 2.7). This turns it on, with a small grid of storage durations and curtailment targets. Each combination is another full optimisation, so the run takes correspondingly longer:

```yaml
eva:
  dummy_storage_optimization:
    enable: true
    energy_power_ratios: [2, 6]
    curtailment_targets: [0.02, 0.05]
```

7.3 Reserves Off, To Isolate Their Effect

Run the same case twice, once with reserves and once without, and compare the indicators. Run the two in separate working directories (see 2.6), or the second run overwrites the first.

```yaml
eva:
  reserves:
    option: none
```

`derated` is the middle option: reserves still constrain generators, but they are not co-optimised with energy, which solves far faster than `co-optimised`.

7.4 Sensitivity Mode

Sensitivity mode builds the base network from a folder of PyPSA component CSVs rather than from a pre-built network, so you can edit the network by hand before solving it. The ACER Paper model already runs in sensitivity mode, so to vary it, copy its folder, edit the copy, and point the model at the copy.

First copy the folder:

```
# Linux and macOS
cp -r data/teaser data/my-teaser
```

```
:: Windows, Miniforge Prompt
xcopy /E /I data\teaser data\my-teaser
```

In PowerShell, use `Copy-Item -Recurse data\teaser data\my-teaser`. Then edit whichever CSVs in `data/my-teaser/ty_2030_cy_2013/` you want to vary, such as capital costs in `generators.csv`, capacities, or the availability timeseries in `generators-p_max_pu.csv`. A spreadsheet program works, as long as you save the file as CSV again with the same columns.

Save this as `config/my-teaser.yaml`:

```
eva:
  build_network:
    sensitivity_path: data/my-teaser/ty_{ty}_cy_{cy}
```

and run it on top of the ACER Paper configuration:

```
snakemake -c all --configfile config/teaser.yaml config/my-
teaser.yaml --directory runs/my-teaser
```

The day-ahead forecasts are still read from `data/teaser/`, as `config/teaser.yaml` names them. To vary them too, also set `eva.build_network.forecast_p_max_pu_path` and `forecast_p_set_path` to the files in your copy.

With pre-built networks instead, the `prepare_sensitivity` target exports the built base networks to CSVs, one folder per target year and climate year under `<checkout>/var/workflow/sensitivity/base_<run.parent>/ty_{ty}_cy_{cy}/` (or under `EUFLEX_SENSITIVITY_DIR`, see 3.21). A run with `eva.sensitivity: true` and `sensitivity_path` left empty then reads them from there.

8 Troubleshooting

8.1 Missing Input Files

```
MissingInputException in rule build_base_network in file "../src/
euflex/eu_flex_model/rules/solve_eva.smk", line 141:
Missing input files for rule build_base_network:
```

```
    output: resources/networks/base_network_2026_2010_forecast_0.nc
    wildcards: ty=2026, cy=2010, ft=forecast, it=0
    affected files:
        /path/to/eu-flex-model/data/pre-builts/base_/input_scenarios/
2026_2010.nc
```

`base_` with nothing after it means the model found no configuration for the run, and fell back on the defaults, which look for pre-built networks that are not part of this release. The usual causes are:

- No `--configfile` was given. Pass the configuration of the run, such as `--configfile config/teaser.yaml`.
- On Windows, in PowerShell, a path in the command starts with `~`. PowerShell does not expand `~` in arguments to programs such as `snakemake`, so `--directory ~/my-run` makes a folder literally named `~` inside the current folder, where Snakemake finds no `config/config.yaml`. Write the path out in full, such as `C:\Users\<name>\my-run`, or use `$HOME/my-run`, which PowerShell does expand.
- The command was not run from the root of the checkout, so a relative `--configfile` path names no file. Snakemake then stops with a message that it cannot find the file; `cd` to the checkout and run it again.

If the missing files are under `data/teaser/`, the model looked for the ACER Paper model in the wrong place. This happens when the working directory is outside the checkout and `EUFLEX_INPUT_DIR` is not set (see 2.6).

8.2 Configuration Rejected at Startup

```
ValidationError: 'derate' is not one of ['none', 'co-optimised',
'derated']
Failed validating 'enum' in schema['properties']['eva']['properties']
['reserves']['properties']['option']:
    ...
On instance['eva']['reserves']['option']:
    'derate'
```

The schema caught a bad value before the run started. The message names the accepted values, and the `On instance[...]` line names the key. This is the cheap way to find configuration mistakes, which is why validation runs first.

8.3 Infeasible Market Windows

An infeasible rolling-horizon window usually means the network cannot physically meet demand under the constraints you set. Things to try, roughly in order:

1. Confirm `solving.options.load_shedding` is `true`. With shedding on, scarcity shows up as an expensive but feasible result rather than a hard failure.
2. Enable `eva.build_network.scale_network` with a `capacity_scaling_factor` above 1, if the first iteration is the one failing.
3. Set `eva.base_network.remove_ramp_su_sd: true`, if start-up ramp limits are cascading.
4. Relax `eva.reserves.option` to `derated` or `none` to find out whether reserves are the binding problem.
5. Export the failing model with `eva.export_lp_models: true` and inspect it.

To keep going and look at the damage instead of stopping, set `eva.fail_on_infeasibility: false`.

8.4 The Run Is Too Slow

In rough order of effect:

1. Set `eva.dummy_storage_optimization.enable: false`, or cut `energy_power_ratios`. Each ratio and target combination is another full optimisation.
2. Reduce `scenario.ty` and `scenario.cy`. Cost scales with the product of the two.
3. Set `eva.reserves.option: derated`. Co-optimising reserves with energy makes the problem considerably bigger.
4. Keep `solving.options.linearized_unit_commitment: true`. Binary unit commitment is orders of magnitude slower.
5. Enable `eva.advanced_clustering` on long runs.
6. Use Gurobi (a commercial solver; see 1.1.3).

8.5 Restarting an Interrupted Run

Snakemake picks up where it left off. Files that already exist are not rebuilt. Pass the same configuration and working directory as before:

```
snakemake -c all --configfile config/teaser.yaml --directory runs/
teaser --rerun-incomplete
```

`--rerun-incomplete` redoes jobs that were killed midway and may have written partial output. If you want a clean slate, delete the run's working directory, for example `runs/teaser`, and start again.

8.6 Running Offline

The workflow downloads nothing. Every input it needs, including the ACER Paper model, is bundled with the release, so a run works the same on a machine with no route to the internet.

If you see a run stall on a network call, it is not the model. The usual causes are conda or pip resolving an environment, or a solver checking out a licence (Gurobi's WLS licences are validated online; HiGHS needs no licence and is the default).

The map plots under `resources/plots/` are drawn from the network's own bus coordinates. They carry no coastline or country backdrop, which is why they need no basemap data and cannot be tripped up by a missing or half-written cache.

8.7 Reproducibility

A run is reproducible: the same configuration, on the same model version, gives bit-identical results with the same solver and solver option profile, whatever else the machine is doing at the time. If two runs you expect to match do not, the cause is a difference in configuration or in input data, not in the model.

Three things do legitimately change results, and are worth ruling out first:

- **The solver.** Different solvers, and different option profiles for the same solver, settle on different solutions where several share the same optimal cost. Results are reproducible only within one `solving.solver` and `solving.solver_options` setting.
- **The input network.** The ACER Paper model and a copy of it edited in sensitivity mode are different networks.
- **Forecast noise.** With `eva.build_network.forecast_timeseries_source` set to `noise`, the day-ahead timeseries are perturbed synthetically. The perturbation is seeded, so it repeats, but it changes if you change the scenario or the pre-built network.

The intermediate networks under `resources/networks/` are NetCDF containers whose bytes include metadata that varies between writes, so `cmp` on two of them reports a difference even when every value in them is equal. Compare their contents rather than their bytes.

8.8 Inspecting What Will Run

With the configuration of your run, as in 2.4:

```
# List the jobs and why each is needed
snakemake -n --configfile config/teaser.yaml --directory runs/teaser

# Draw the dependency graph
snakemake --dag --configfile config/teaser.yaml --directory runs/
teaser | dot -Tpng -o dag.png
```

Drawing the graph needs Graphviz, which provides `dot` and is not part of the model's environment. Install it with your system package manager or with `conda install -c conda-forge graphviz`. Alternatively, `snakemake --dag mermaid-js` prints the graph in Mermaid format, which GitHub and many editors render directly.

---

9 Feedback

We welcome feedback on the EU-Flex model implementation and documentation. Your input helps us improve the model and better serve the research community.

> **Feedback Deadline**: The feedback collection period for this pre-release closes on **December 17th, 2026**. Issues submitted after this date will not be considered for this release cycle.

How to provide feedback

This repository uses GitHub Issues to collect feedback on:

- **Bugs**: unexpected behaviour, errors, or crashes
- **Feature requests**: suggestions for improvements or new functionality
- **Questions**: clarification on model behaviour or usage
- **Documentation feedback**: feedback on the README and User Manual

To submit feedback:

1. Navigate to the Issues page
2. Click **New Issue**

3. Select the appropriate template:
    - **Bug Report** for issues with the code or unexpected behaviour
    - **Feature Request** for suggestions and enhancements
    - **Question** for clarification or help with usage
    - **Documentation Feedback** for feedback on the README and User Manual
4. Fill out the template with as much detail as possible
5. Submit the issue

Your feedback will be reviewed and addressed by the development team. We aim to respond to all issues in a timely manner.

What feedback is collected here

This feedback mechanism focuses on **technical implementation** aspects:

- Code implementation and model behaviour
- Documentation clarity and completeness
- User experience with the model

For feedback on the **methodology** (the modelling specification) or research outputs (the ACER Paper), please refer to the ACER survey mechanism communicated separately.

---

10 Licence and Contributions

The code and model configuration are released under the MIT licence, the documentation and the ACER Paper model under CC-BY-4.0, and repository settings, environment files and other input data under CC0-1.0. Some of the code is adapted from PyPSA-Eur and keeps its copyright notice. The licence texts are in the `LICENSES/` directory, and `REUSE.toml` and the file headers record which applies to which file, following the REUSE specification.

© Copyright Open Energy Transition.
Created using Sphinx 7.4.7.